



Defence Institute of Advanced Technology

MID-SEM EVALUATION PRESENTATION



Offline Agentic AI Application for SDLC

PRESENTED BY

Abhishek Raj (25-26-15)

GUIDES

Prof. CRS Kumar

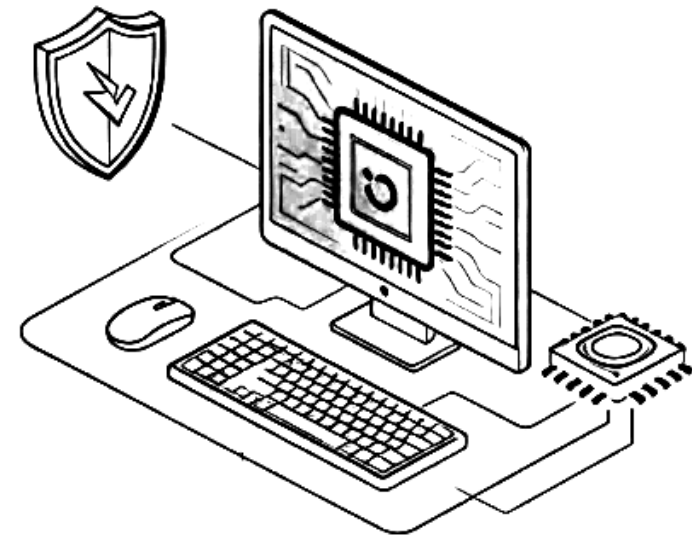
M.B. Sai Charan (Scientist C, RCI DRDO)

Problem Statement

Defence organizations operate under strict data security mandates that prohibit exposing proprietary source code to third-party, cloud-based Large Language Models (LLMs). To safely leverage AI in engineering, there is a critical need for an autonomous Software Development Lifecycle (SDLC) framework that operates entirely offline, ensuring all code, data, and execution remain secured within air-gapped local infrastructure.

Local execution ensures data privacy but introduces **compute and context limitations**. Existing multi-agent SDLC systems rely on powerful cloud infrastructure and large-context LLMs, making them difficult to deploy on constrained hardware such as a **single GPU**.

Feeding entire repositories to local LLMs causes **memory exhaustion, high latency, context dilution, and hallucinations**. Hence, the challenge is to build an **offline, resource-efficient, repository-aware agentic SDLC framework** that retrieves only task-relevant context while autonomously coordinating development activities.



Literature Review

To solve the local deployment bottlenecks, we first analyzed the theoretical foundations and limitations of current AI workflows to understand exactly what our architecture needed to fix.

- **LLM-Based Multi-Agent Systems for Software Engineering: Literature Review, Vision, and the Road Ahead (He, J., Treude, C., & Lo, D., 2025): (Multi-Agent Task Delegation):** Establishes the necessity of dividing SDLC workflows among specialized agents. It identifies long-term memory and context limits as the primary blockers for enterprise adoption, validating our need for a token-efficient project graph.
- **Assistance to Autonomy: A Systematic Literature Review of Agentic AI across the SDLC (Apostolou, S. A., et al., 2026): (The Verifiability Principle) :** Highlights that software automation only works if the outputs can be systematically verified. The paper notes that current agents lack robust evaluation harnesses, driving our integration of a self-correcting Reflexion loop.
- **Agentic AI in the Software Development Lifecycle: Architecture, Empirical Evidence, and Operational Mechanics (2026): (Empirical Failure Benchmarks):** Real-world benchmarking reveals that agents quickly lose architectural consistency during extended sessions. This proved that standard generative prompting is insufficient, pointing us directly toward isolated, surgical code editing.
- **The Hitchhiker's Guide to Agentic AI: From Foundations to Systems(Roitman, H., 2025) : (Agentic Harness Engineering) :** Provides the theoretical blueprint for separating an LLM's raw reasoning from its physical execution environment. We translated these concepts—like state management and loop engineering—into our offline LangGraph pipeline.
- **Codebase-Memory: Tree-Sitter-Based Knowledge Graphs for LLM Code Exploration via MCP (2026): Tree-Sitter Knowledge Graphs:** Demonstrates how to map entire repositories without exceeding token limits by parsing Abstract Syntax Trees (AST) and scoring dependencies. This methodology directly inspired the structural foundation of our Hierarchical Software Knowledge Graph (HSKG).

THE CHALLENGE



The Context Window Bottleneck

Local models cannot process massive amounts of code at once. Flooding their limited context window with full repository text leads to high latency, severe memory degradation, and logic hallucinations.



Absence of Topological Memory

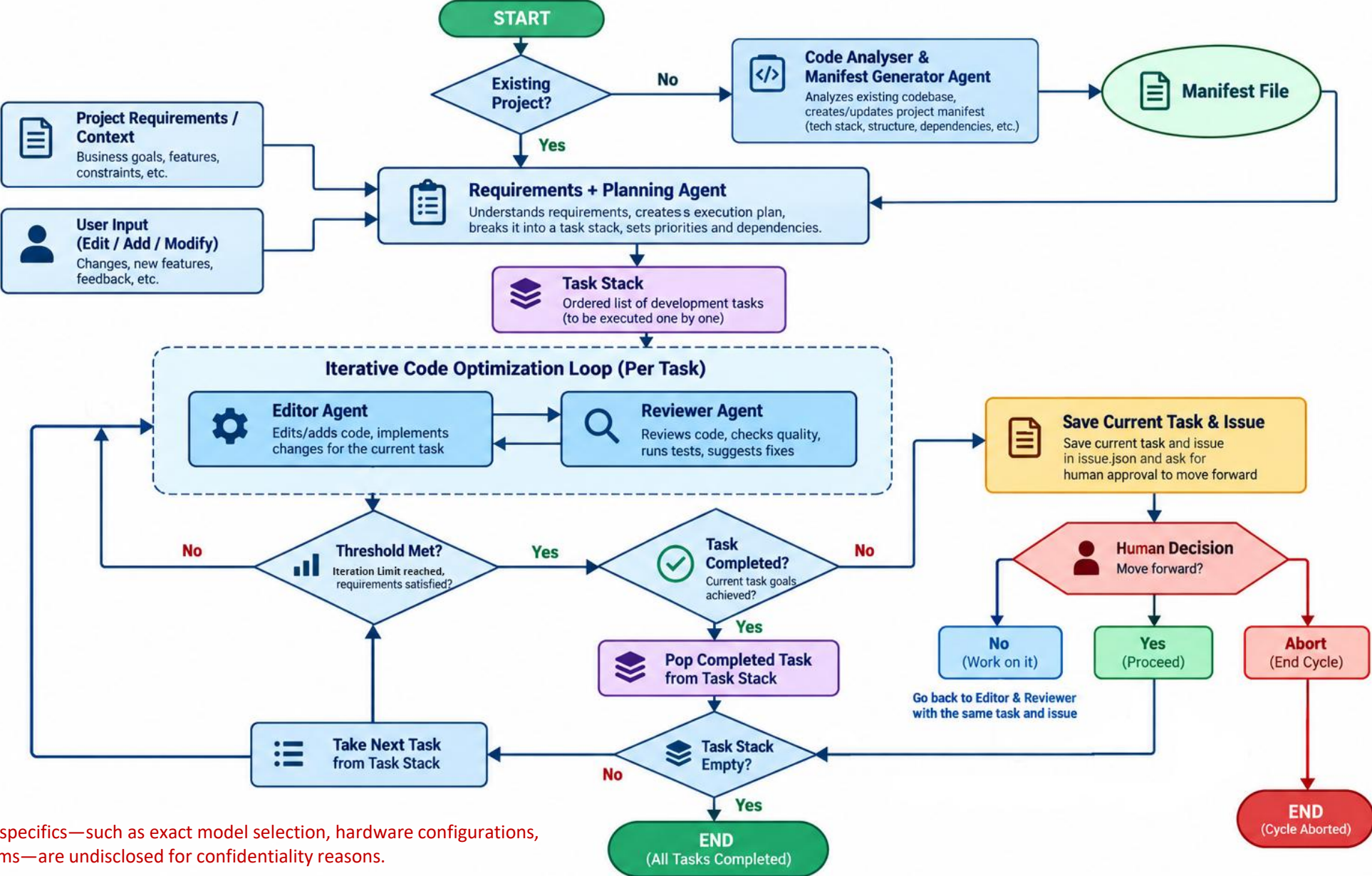
Standard (RAG) techniques rely on flat keyword searches, completely lacking a real structural understanding of the software project. Because of this, the AI frequently misses distant structural dependencies. For example, it might fail to recognize how a backend database update breaks a frontend controller, how modifying a core utility function cascades silent failures across dozens of unrelated modules, or how altering an API response payload in one microservice crashes a downstream consumer that lacks textually similar keywords.



Risks of Generative Rewriting:

To implement even minor feature requests, simple llm harness tend to rewrite entire files from scratch. This approach is highly inefficient for local models and introduces a serious risk of accidentally deleting perfectly working code.

Proposed Architecture: Modular Pipeline



Note: Certain technical specifics—such as exact model selection, hardware configurations, and underlying algorithms—are undisclosed for confidentiality reasons.

Approach: Hierarchical Software Knowledge Graph (HSKG)

Beyond Traditional Memory:
A Context-Aware Project Memory

The Problem: Current AI agents retrieve entire repositories or large documents, resulting in high token consumption and irrelevant context.

The Solution: The **Code Analyser and Manifest Generator Agent** creates **HSKG**, also known as the **Manifest** of the project, that stores the software project as a hierarchical graph, **enabling agents to track & retrieve** only the smallest relevant subgraph required for a task.



Project Graph

Project

- └ Authentication
 - Login, Register
- └ Inventory
 - Product, Stock
- └ Invoice
 - Create, Payment

Purpose

- Represents the complete SDLC structure
- Maintains hierarchy between project components
- Enables modular navigation



Context-Aware

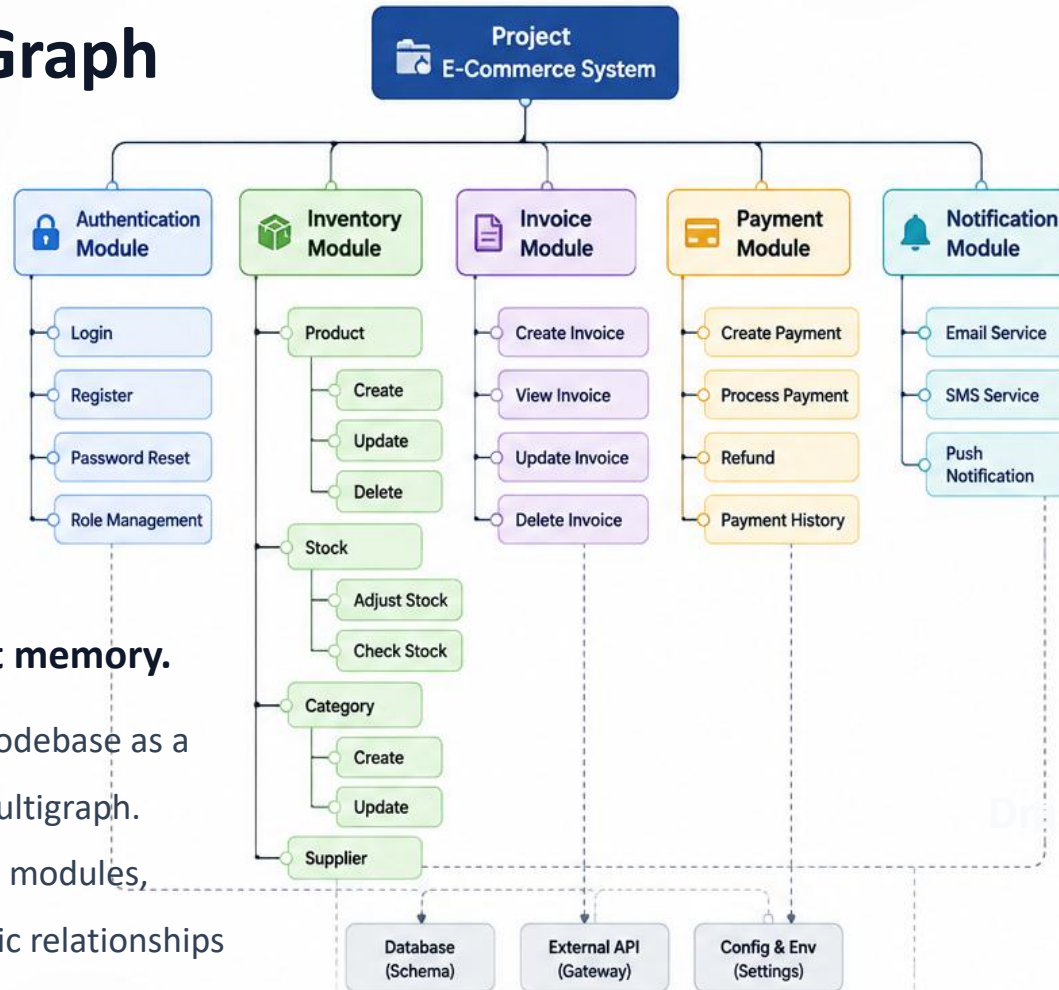
Instead of:
Entire Repository → LLM

Agent retrieves:
Login Module → UI → Backend Script & Variables
→ API → User DB

Benefits

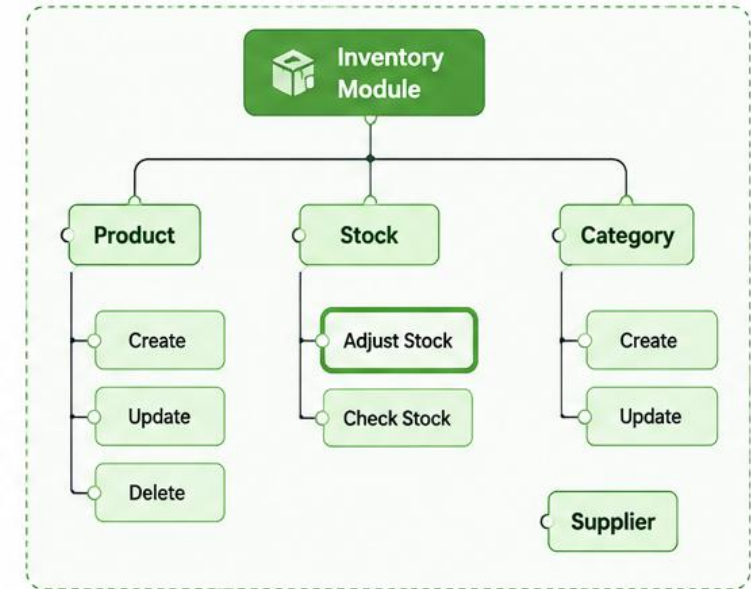
- Minimal context
- Lower token usage
- Faster reasoning
- Better accuracy
- Reduced hallucination

Hierarchical Software Knowledge Graph



Agent Query:
"How is stock adjusted when a new order is placed?"

Subgraph Retrieval: Selected Area (Inventory Module)



Context Provided to Agent



Other parts of the project are not sent to the agent

- ✓ Relevant nodes & relations
- ✓ Minimal context
- ✓ Lower token usage
- ✓ Faster & accurate reasoning

✗ Irrelevant modules excluded (Not sent to LLM)

Context-aware project memory.

- **AST Parsing:** Models codebase as a directed, attributed multigraph.
- **Nodes & Edges:** Tracks modules, functions, and semantic relationships (calls, modifies).
- **Topology:** Retrieves strict k-hop subgraphs instead of flat text files.

Stack-Based Editor Loop

Reducing context overhead through task decomposition, localized editing, and iterative verification.

01. Requirement & Planner Agent

Breaks the user's large request into a "stack" of bite-sized, step-by-step tasks. By forcing the AI to deal with only one small task at a time, we completely avoid overwhelming the local model's limited memory.

02. Editor

This Agent retrieves the highest-priority task from the stack and isolates only the necessary code dependencies using the topological graph. Instead of rewriting full files, it executes surgical **SEARCH/REPLACE block edits** (diff patching). This targeted approach drastically reduces local compute latency and completely safeguards untouched, functioning code from unintended AI hallucinations.

03. Reviewer

Checks the newly patched code against the project requirements to ensure it actually works. If it spots a mistake, it sends an error report right back to the Editor so the AI can learn from it and fix it immediately.

Additional Harness Engineering

For making the A-SDLC pipeline observable, adaptive, and fail-safe

DeepEval — Continuous Agent Evaluation

We are integrating DeepEval (an open-source evaluation framework) into the agent workflow to evaluate **individual agents, nodes, iterations, tool calls, and complete trajectories** across metrics such as task success, correctness, relevance, faithfulness, tool use, and efficiency. This provides continuous evaluation rather than checking only the final output.

Loop Engineering — Learn, Correct & Recover

The execution loop incorporates **Reflexion-style learning**, where failures and reviewer feedback are converted into reflections and used to improve subsequent iterations. Each task is repeatedly **execute → evaluate → reflect → correct → re-evaluate**, while iteration limits, error thresholds, and explicit termination conditions provide **fail-safe termination** when progress stalls or unsafe behavior is detected.

Implementation & Preliminary Results

Implementation Tracks

1. Demonstration Prototype

We built a functional, simplified workflow to prove the concept. It successfully takes a user request and autonomously generates complete mini web applications (HTML/JS/CSS).

2. Main Project (In Progress)

For real-world codebases, we are actively developing the core Analyzer node. This parses the repository to create the Knowledge Graph and builds a visual UI so humans can understand the AI's map.

Results & Focus

Core Planner Node Built: The foundational logic that acts as our "Analyzer Node" is complete. The system can now analyse the entire repo and form the knowledge graph of it as well as receive a user request and attempt to break it down into a step-by-step task stack.

Active Optimization: Our immediate priority is fine-tuning this Analyzer node to hit a strict accuracy threshold before finalizing the Editor and Reviewer nodes.

Current Engineering Goals:

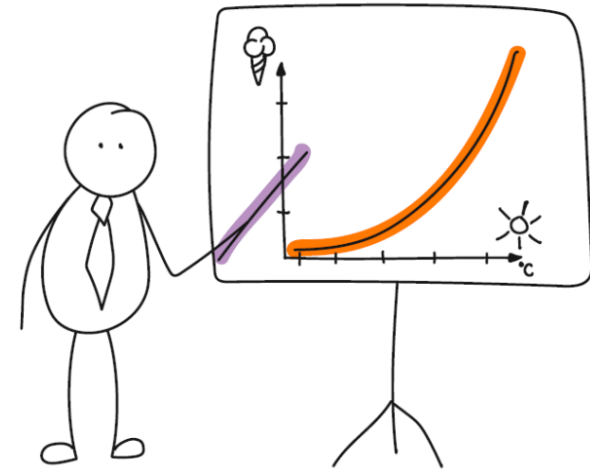
- **Accurate Repository Mapping:** Ensuring the AI perfectly reads and understands the existing codebase without missing context.
- **Flawless Task Stacks:** Training the Analyzer to generate a completely logical, ordered stack of edits that doesn't break hidden dependencies.

Conclusion & Future Scope

The integration of the Hierarchical Software Knowledge Graph (HSKG) and stack-based execution successfully addresses the memory and context limitations of local, offline LLMs.

Moving toward the end-semester evaluation, our primary focus is stabilizing the workflow to ensure the generated code patches are highly accurate and functionally coherent.

Additionally, we will fully automate the codebase mapping using Tree-sitter for native AST parsing, finalize a visual dashboard for human oversight, and integrate Agent-to-Agent (A2A) protocols to seamlessly delegate testing and deployment tasks to isolated containerized agents.



Key References

1. He, J., Treude, C., & Lo, D. "LLM-based multi-agent systems for software engineering: Literature review, vision, and the road ahead," *ACM Trans. Softw. Eng. Methodol.*, 2025.
2. Apostolou, S. A. et al., "Assistance to autonomy: A systematic literature review of agentic AI across the SDLC," *arXiv preprint*, 2026.
3. Anonymous et al., "Agentic AI in the software development lifecycle: Architecture, empirical evidence, and operational mechanics," *arXiv preprint*, 2026.
4. Anonymous et al., "Codebase-Memory: Tree-Sitter-Based Knowledge Graphs for LLM Code Exploration via MCP," *arXiv preprint*, 2026.
5. Roitman, H., *The Hitchhiker's Guide to Agentic AI: From Foundations to Systems*, 1st ed. New York, NY: TechPress, 2025.



Thank You

Questions & Discussion